

Hibernate ddl-auto Guide

Hibernate `ddl-auto` — Complete Guide

Overview

The `spring.jpa.hibernate.ddl-auto` property controls how Hibernate manages your database schema at application startup. Depending on the value, Hibernate can validate, update, create, or drop your database tables automatically based on your JPA entity classes.

1. `validate`

What it does: Hibernate compares your entity mappings against the existing database schema and checks that they match. It does **not** change the database in any way.

How it works: On startup, Hibernate reads the schema of the connected database and verifies that every table and column required by your `@Entity` classes already exists with compatible types. If something doesn't match (a missing column, a wrong data type, a missing table), the application fails to start and throws an exception.

Simple example:

```
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double price;
}
```

```
spring.jpa.hibernate.ddl-auto=validate
```

If the `product` table doesn't already have `id`, `name`, and `price` columns, the application will refuse to start.

Real-world case: Used in **production environments**. Schema changes are handled separately through migration tools like **Flyway** or **Liquibase**, and `validate` is used only to make sure the application code and the real database are in sync — with zero risk of Hibernate accidentally modifying production data.

2. update

What it does: Hibernate compares the entity mappings with the existing schema and applies the **missing changes only** — it adds new tables and new columns, but it never deletes existing ones or removes old columns.

How it works: At startup, Hibernate inspects the current schema. If a table or column required by an entity is missing, Hibernate generates and executes the necessary `ALTER TABLE` or `CREATE TABLE` statements. Existing data is preserved, and Hibernate will not remove a column or table just because it disappeared from the entity class.

Simple example:

```
spring.jpa.hibernate.ddl-auto=update
```

If you add a new field to the `Product` entity:

```
private String description;
```

Hibernate will run something like:

```
ALTER TABLE product ADD COLUMN description VARCHAR(255);
```

Existing rows keep their data; the new column is simply added (usually as `NULL` for old rows).

Real-world case: Common during **local development** and **early-stage / demo projects**, where the schema changes frequently and developers want Hibernate to keep the database in sync automatically without writing manual SQL. It's generally **not recommended for production**, because it can miss complex changes (like renaming a column, which Hibernate sees as "add new + leave old behind").

3. create

What it does: Hibernate **drops all existing tables** (if they exist) and **creates them from scratch** every time the application starts, based purely on the entity classes.

How it works: On startup, Hibernate first deletes any tables mapped by your entities (if present) and then generates fresh `CREATE TABLE` statements matching the current entity definitions. All previously stored data is lost.

Simple example:

```
spring.jpa.hibernate.ddl-auto=create
```

Every time you restart the application, the `product` table is deleted and recreated empty, even if it had 1,000 rows before.

Real-world case: Useful in **early development or prototyping**, when the data model is still changing rapidly and you don't care about losing existing data on every restart — for example, when building a proof-of-concept or testing a fresh entity structure.

4. create-drop

What it does: Same as `create` (tables are dropped and recreated at startup), but with one extra behavior: Hibernate also **drops all the tables again when the application shuts down**.

How it works: On startup, Hibernate creates the schema fresh from the entities. When the Spring context closes (application stops), Hibernate automatically executes `DROP TABLE` statements, leaving the database completely clean afterward.

Simple example:

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Run your test suite → tables are created → tests run → application shuts down → tables are dropped automatically. No manual cleanup needed.

Real-world case: The standard choice for **automated testing** (unit/integration tests) and **temporary in-memory databases** like H2. It guarantees a clean, isolated schema for every test run without leaving leftover data behind.

5. none

What it does: Hibernate does **absolutely nothing** to the database schema. No validation, no creation, no updates.

How it works: Hibernate simply uses the schema exactly as it already exists in the database and trusts that it matches the entity mappings. It performs zero DDL (Data Definition Language) operations at startup.

Simple example:

```
spring.jpa.hibernate.ddl-auto=none
```

Even if the `product` table is missing a column that the `Product` entity expects, Hibernate won't complain at startup — the error will only appear later, when a query actually tries to use that column.

Real-world case: The recommended setting for **production systems** in many enterprise setups, where all schema management is done exclusively through dedicated migration tools (Flyway, Liquibase) or DBA-controlled scripts, and Hibernate is intentionally kept out of DDL responsibilities entirely.

🔥 Detailed Walkthrough: update

```
spring.jpa.hibernate.ddl-auto=update
```

What does Hibernate do to the database? It scans the current schema, compares it to the entity classes, and adds whatever is missing — new tables, new columns — without touching or deleting existing data or columns that are no longer mapped in the code.

Real-world case for update: A startup team building an MVP where the data model changes daily. Instead of writing and running manual migration scripts for every small change, the team lets Hibernate auto-adjust the development database schema, saving time during the early, fast-iteration phase of the project — while switching to `validate` or `none` once the app moves to production.

☆ Summary Comparison Table

Value	What it does	Real-world Case
validate	Checks that entities match the existing schema; makes no changes; fails startup on mismatch	Production environments where schema is managed by Flyway/Liquibase and Hibernate must not touch the DB
update	Adds missing tables/columns automatically; never deletes existing ones	Local development / early-stage projects with a fast-changing data model
create	Drops all existing tables and recreates them from entities on every startup	Prototyping / proof-of-concept work where losing data on restart is acceptable
create-drop	Creates schema on startup, then drops it again when the app shuts down	Automated tests and in-memory databases (e.g., H2) needing a clean slate each run
none	Performs no schema action at all; Hibernate relies entirely on the existing DB structure	Production systems where DBAs/migration tools fully own schema changes

⚠️ Important Note

`ddl-auto` values other than `validate` and `none` are considered **unsafe for production** because they let Hibernate automatically alter or destroy real data. Best practice: use `update/create/create-drop` only in development/testing, and switch to `validate` or `none` combined with a proper migration tool (Flyway/Liquibase) in production.